Milestone #2 – 4+1 View Model

BabyTrack+

Project Purpose

The purpose of **BabyTrack+** is to monitor and log a baby's daily activities, including feeding times, sleep schedules, diaper changes, and health observations. The system helps caregivers track routines, identify patterns, and generate simple reports to support consistent and informed caregiving.

The application stores data in a cloud database, allowing caregivers to access records across devices.

Target Audience

Primary Users

- Parents and primary caregivers
- Family members assisting with childcare
- Daycare providers managing infant schedules

Requirements

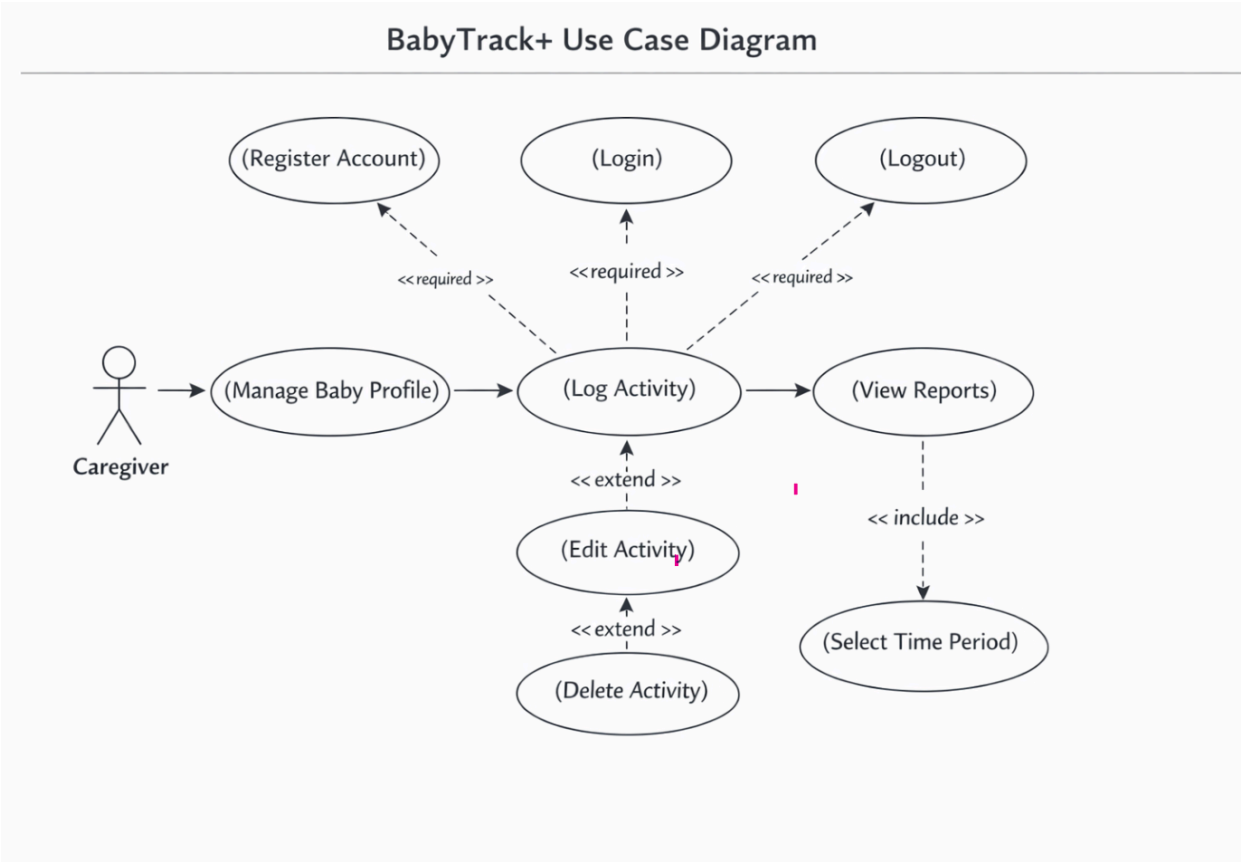
1. The system shall allow users to create an account and securely log in using a unique username and password.
2. The system shall allow users to create, view, update, and delete baby profiles associated with their account.
3. The system shall allow users to log daily activities for each baby, including feeding, sleep, diaper changes, and health notes.
4. The system shall generate summary reports for selected time periods, displaying aggregated activity data in a clear and readable format.
5. The system shall store and retrieve all user and activity data using a cloud-based MySQL database.

6. The system shall respond to user actions within 2 seconds under normal operating conditions

Scenario Viewpoint

Use Case Diagram

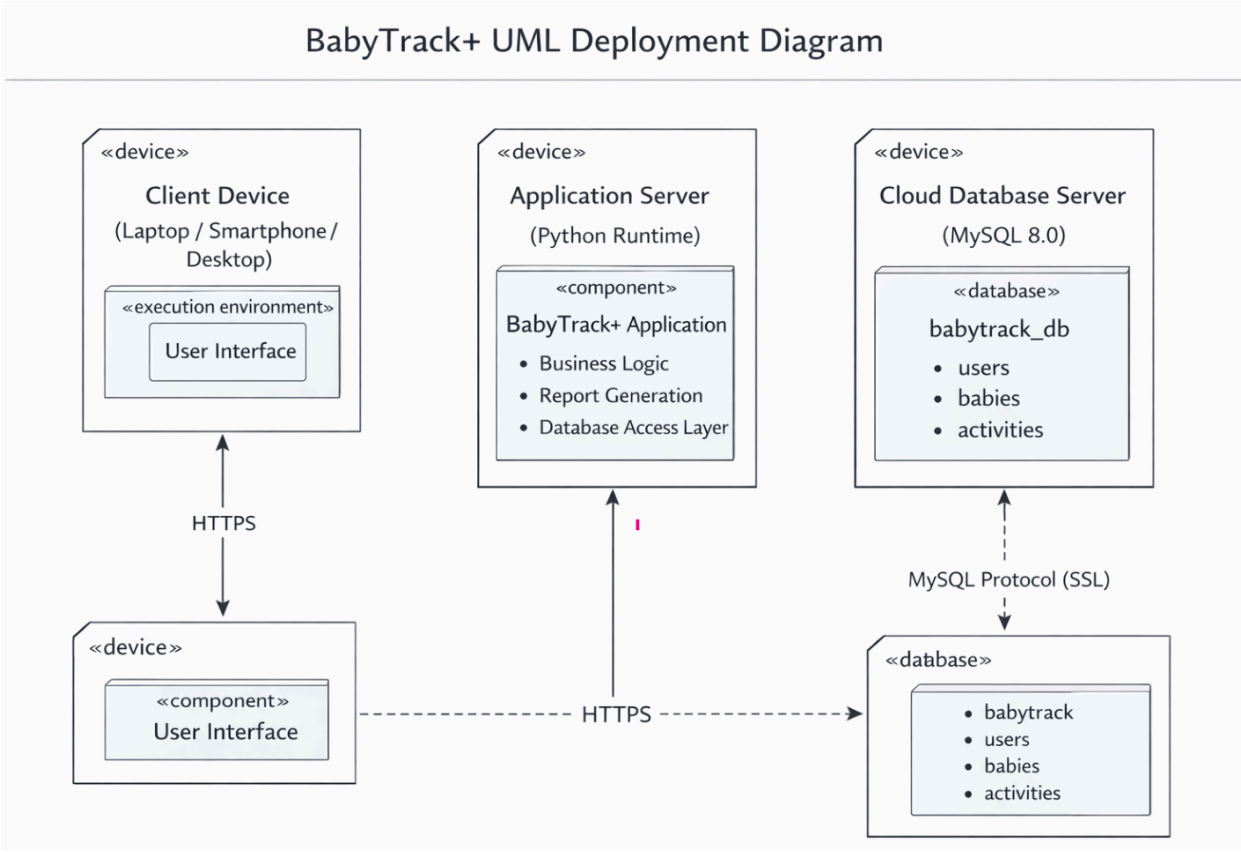
The primary actor is the **Caregiver**. The caregiver can register, log in, and log out of the system. After authentication, the caregiver can manage baby profiles, log activities, edit or delete existing activity records, and generate reports for selected time periods. The "View Reports" use case includes selecting a time range before generating the summary.



Physical Viewpoint

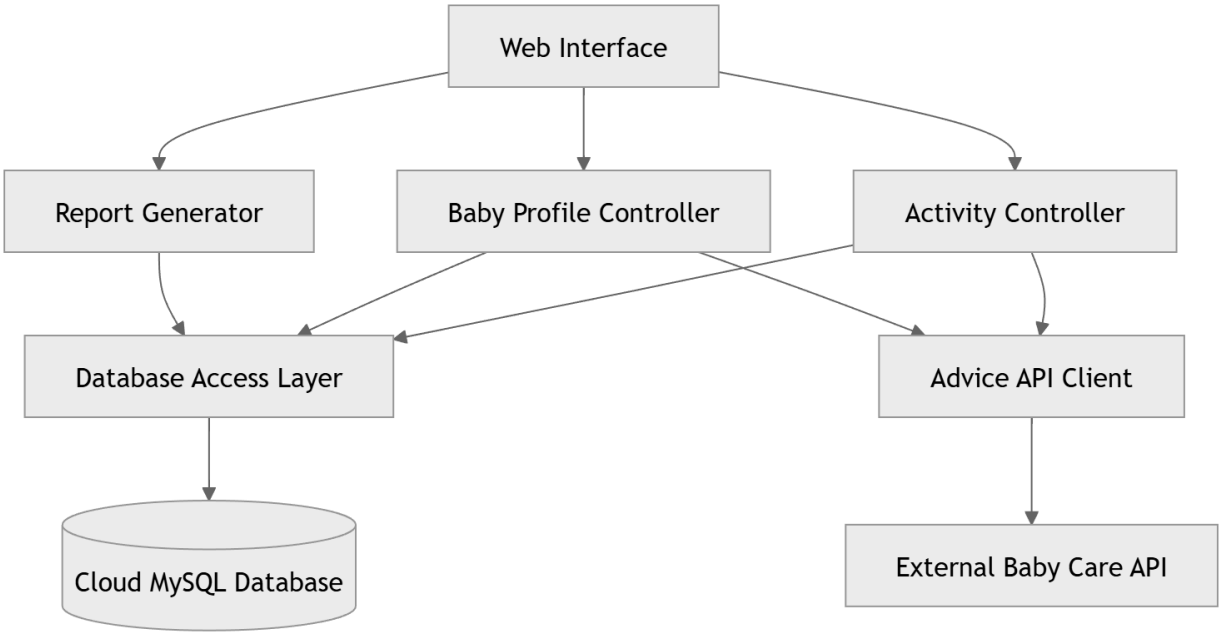
UML Deployment Diagram

The Physical Viewpoint is represented using a UML deployment diagram that follows a three-tier architecture. The system consists of a Client Device (user interface), an Application Server running the BabyTrack+ application and business logic, and a Cloud Database Server using MySQL for persistent storage. Communication between components occurs securely using HTTPS and MySQL over SSL.



Development Viewpoint

UML Component Diagram



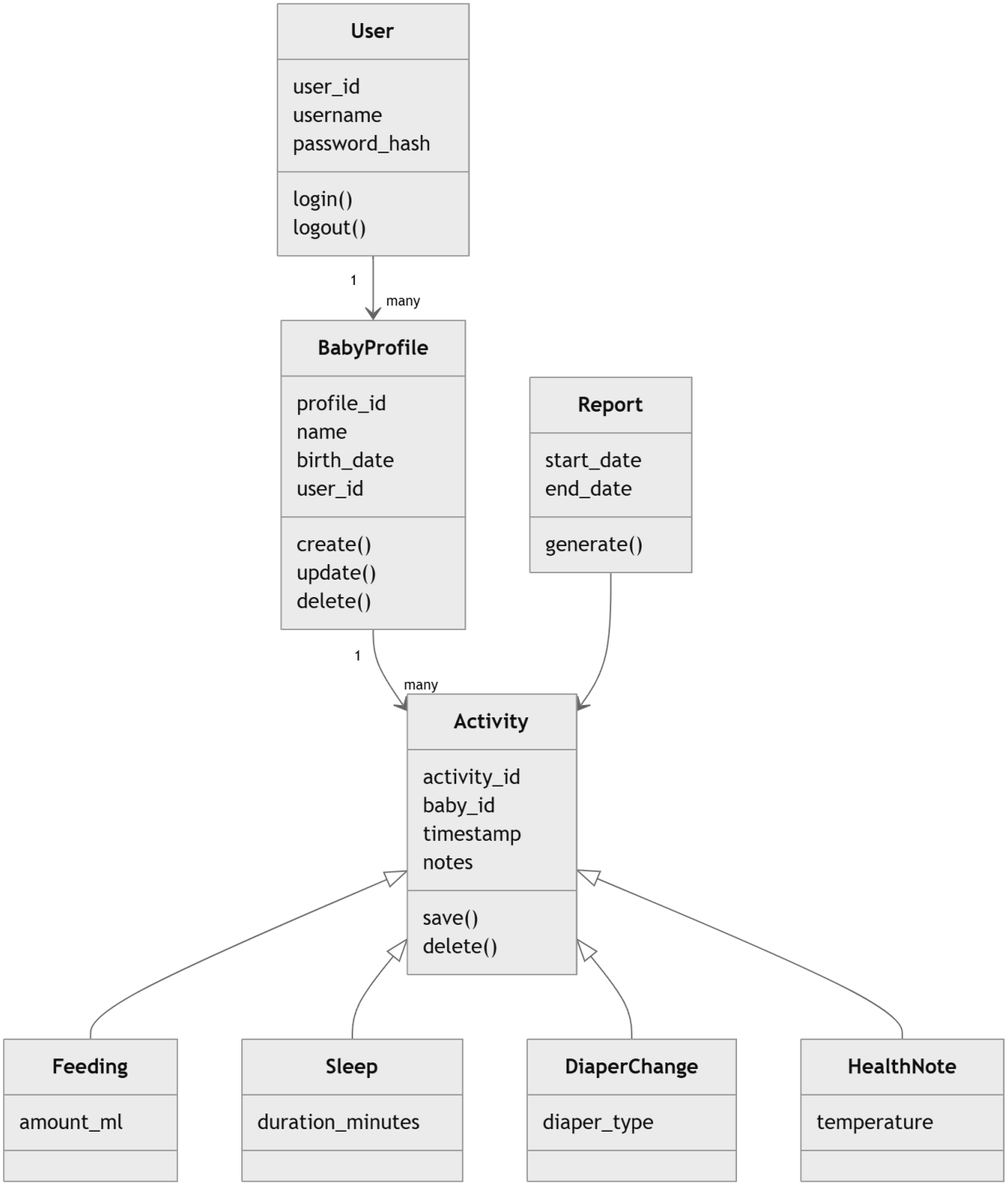
The BabyTrack+ system is divided into several major components:

- **Web Interface** – Handles user interaction and collects input from caregivers.
- **Activity Controller** – Manages operations related to activity logging such as feeding, sleep, diaper changes, and health notes.
- **Baby Profile Controller** – Handles CRUD operations for baby profiles. **Report Generator** – Aggregates activity data and produces summary reports for selected time periods.
- **Database Access Layer** – Encapsulates all SQL operations and database connections.
- **Advice API Client** – Communicates with an optional external baby-care advice service.
- **Cloud MySQL Database** – Stores all persistent user, baby, and activity data.

This modular structure improves maintainability, separation of concerns, and scalability, allowing different components to be developed or updated independently.

Logical Viewpoint

UML Class Diagram



The Logical Viewpoint focuses on the core domain model and object relationships within the system.

The main classes in BabyTrack+ include:

- **User** – Represents caregivers who use the system.
- **BabyProfile** – Stores information about each baby being tracked.
- **Activity** – Represents logged baby activities such as feeding or sleeping.
- **Report** – Generates summaries of activity data.

The Activity class serves as a base class, with specialized subclasses:

- Feeding
- Sleep
- DiaperChange
- HealthNote

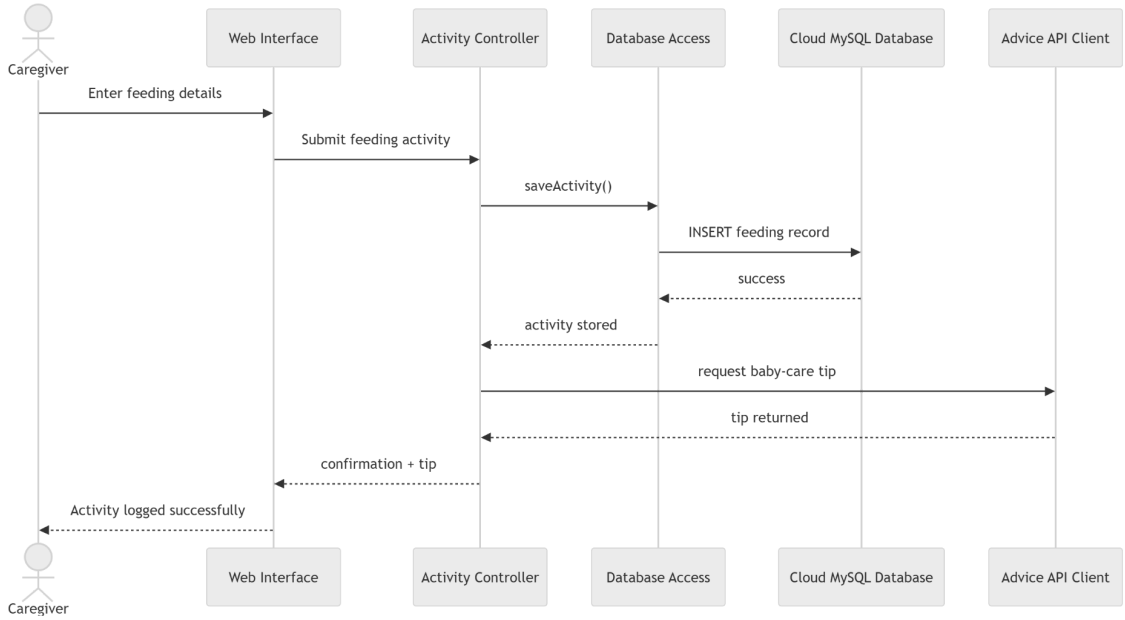
A User can manage multiple BabyProfiles, and each BabyProfile can contain multiple Activities.

This object-oriented structure improves extensibility. For example, new activity types could be added later without modifying existing classes.

Process Viewpoint

UML Sequence Diagram

Scenario: **Caregiver logs a feeding activity**



The Process Viewpoint describes the runtime interaction between system components during execution.

In this scenario:

1. The Caregiver enters feeding information through the Web Interface.
2. The interface sends the request to the Activity Controller.
3. The controller calls the Database Access layer to store the activity.
4. The data is inserted into the Cloud MySQL database.
5. After successful storage, the system optionally requests advice from an external baby-care API.
6. The system returns a confirmation message and displays it to the caregiver.

This sequence illustrates the order of operations and data flow between components.